Evaluation Type: Lab 2: Index Based Data Structures

Details

Weighting: 2%
Submission Instructions: This will be reviewed in class and submitted as part of your lab portfolio
Due Date: Thursday, September 17

Overview

In previous labs, you have worked with **arrays** to store multiple values or objects under a single variable name. You have accessed individual elements using an index, processed arrays with loops, and seen that an array's size is fixed when it is created.

Arrays are fast, simple, and useful, but they also introduce some limitations. If an array becomes full, you cannot simply add another element. If an element is removed from the middle of an array, you may need to move other elements to close the gap. You may also need to keep track of how many positions in the array are actually being used.

In this lab, you will build on your knowledge of arrays by exploring **index-based data structures** that make these operations easier.

You will begin by using Java's built-in **ArrayList** class. Like an array, an ArrayList stores elements in an ordered collection and allows them to be accessed by index. Unlike a regular array, however, an ArrayList can grow and shrink as elements are added and removed.

You will then look behind the curtain and build your own **ArrayManager** class. Your ArrayManager will use an ordinary array internally, but will provide methods that perform some of the same operations offered by an ArrayList. This will require you to manage details such as the current number of elements, available capacity, and what happens when elements are added or removed.

By moving from arrays, to ArrayList, and then to your own ArrayManager, you will see how familiar array concepts can be used to construct a more flexible data structure.

Skills Summary

By the end of the lab, you should have a better understanding of:

- how arrays and ArrayList are similar and different;

- how index-based collections organize and access data;
- how the logical size of a collection differs from the capacity of its underlying array;
- what happens behind the scenes when elements are added or removed;
- how to implement common operations such as add, get, set, and remove; and
- how higher-level data structures can be built using arrays as their foundation.

Instructions

Part 1: Working with ArrayList

An array stores items in indexed positions, but its size is fixed when the array is created. Java's ArrayList provides a similar index-based structure while also allowing items to be added and removed.

In this exercise, you will use an ArrayList<Integer> to practise common operations before implementing similar functionality yourself in the ArrayManager class.

Instructions

Create an ArrayList<Integer> and add the following values to it:

12, 7, 25, 4, 18

Perform each of the following operations:

1. Display the number of items in the ArrayList.
2. Use a loop to display all of the items.
3. Add the value 30 to the end of the list.
4. Insert the value 15 at index 2.
5. Remove the item at index 4.
6. Display the item stored at index 3.
7. Change the item at index 0 to 100.
8. Display the completed ArrayList.

Your output should clearly show the list after the add, insert, remove, and update operations.

Part 2: Building an ArrayManager

Java's ArrayList provides methods for adding, removing, and accessing elements while managing the underlying storage for you. In this part of the lab, you will create your own simplified version of this functionality using an array.

The ArrayManager Class

Create a class called ArrayManager with the following instance variables:

```
private int[] items;

private int count;
```

The items array will provide the physical storage for the integers in the collection. The count variable will keep track of how many positions in the array are currently in use.

For example, an ArrayManager may have an items array with a capacity of 10, but a count of only 4.

Constructors

Create the following constructors:

- **`ArrayManager()`**
  Creates an empty ArrayManager. Set count to 0 and create an items array with a capacity of 10.
- **`ArrayManager(int capacity)`**
  Creates an empty ArrayManager with an items array of the specified capacity. Set count to 0.
- **`ArrayManager(int[] values)`**
  Creates an ArrayManager initialized with the integers contained in values. Set count appropriately.

Methods

Add the following methods to the ArrayManager class:

`public int size()`

Returns the number of items currently stored in the ArrayManager.

`public void print()`

Displays all items currently stored in the ArrayManager.

`public void add(int n)`

Adds n to the end of the items currently stored in the ArrayManager.

`public void remove(int pos)`

Removes the item at index pos. Any items appearing after the removed item must be shifted one position to the left so that there are no gaps in the collection.

`public void addAt(int n, int pos)`

Adds n at index pos. Existing items at that position and later positions must be shifted one position to the right to make room for the new value.

A valid insertion position is between 0 and count, inclusive. Adding at position count is equivalent to adding the item to the end of the collection.

`public boolean isEmpty()`

Returns true if the ArrayManager contains no items and false otherwise.

Exceptions

Create the following custom exception classes.

`NoItemsException`

Create a class called NoItemsException that extends Exception.

If remove() is called when the ArrayManager is empty, the method must throw a NoItemsException.

`OutOfBoundsException`

Create a class called OutOfBoundsException that extends Exception.

If addAt() is called with a position outside the valid range of positions currently available in the collection, the method must throw an OutOfBoundsException.

For example, if count is 4, the valid positions for addAt() are:

0, 1, 2, 3, 4

Position 4 adds the new item to the end of the collection.

Driver Program

Create a separate driver program to test your ArrayManager.

Begin by creating an ArrayManager containing **9 integers**.

The program should repeatedly display the following menu:

1. Display number of items
2. Display all items
3. Add an item
4. Add an item at a position
5. Remove an item
6. Exit

Use the appropriate ArrayManager method to perform each operation.

Your driver program should also catch and display an appropriate message for any NoItemsException or OutOfBoundsException thrown by the ArrayManager.